ABSTRACT

College administration still depends heavily on paper registers, notice boards, scattered spreadsheets and messaging groups. Attendance is marked and totalled manually, marks and timetables reach students late, notices are missed, and leave or on-duty requests move slowly between students, faculty and the department office. This wastes the time of students, faculty and administrative staff alike.

This project presents **Campus Connect**, a web-based Digital College Management System that brings the day-to-day academic and administrative activities of a college onto a single platform. Students can securely log in to view their profile, timetable, attendance, internal marks, assignments, notices and fee status, and can apply for leave or on-duty and raise grievances. Faculty can mark attendance, upload marks and assignments, publish notices and approve student requests. Administrators manage users, departments, courses, timetables and announcements and view summary dashboards and reports.

The prototype is developed using HTML5, CSS3 and JavaScript on the front end, and Python with the Flask framework and an SQLite database on the back end. Passwords are stored as hashes and access is controlled by role (student, faculty, administrator). The system reduces paperwork, improves transparency, speeds up communication and gives the institution organised, reliable data for decision making.

Keywords: Campus management, digital college, attendance, role-based access, notices, leave management, Flask, SQLite.

TABLE OF CONTENTS

BONAFIDE CERTIFICATE ..... **Error! Bookmark not defined.**
ABSTRACT 1
TABLE OF CONTENTS 2
LIST OF ABBREVIATIONS 4
CHAPTER 1 – INTRODUCTION 5
1.1 Background 5
1.2 Overview of the Project 5
1.3 Need for the Project 5
1.4 Scope of the Project 5
1.5 Organization of the Report 6
CHAPTER 2 – PROBLEM STATEMENT AND OBJECTIVES 7
2.1 Problem Statement 7
2.2 Objectives 7
CHAPTER 3 – EXISTING SYSTEM AND PROPOSED SYSTEM 8
3.1 Existing System 8
3.2 Proposed System 8
3.3 Comparison 8
CHAPTER 4 – SYSTEM REQUIREMENTS 9
4.1 Functional Requirements 9
4.2 Non-Functional Requirements 9
4.3 Hardware and Software Requirements 9
CHAPTER 5 – SYSTEM DESIGN 11
5.1 Solution Design 11
5.2 System Architecture 11
5.3 Workflow 11
5.4 Module Description 12
5.5 Key Features 12
CHAPTER 6 – IMPLEMENTATION 13
6.1 Technology Stack 13
6.2 Implementation Details 13
6.3 Student Side 13
6.4 Faculty Side 14
6.5 Administrator Side 14
6.6 AI and Smart Features (Proposed) 14
CHAPTER 7 – RESULTS AND OUTPUT SCREENS 15

CHAPTER 8 – TESTING....20
8.1 Testing Approach....20
8.2 Test Cases....20
CHAPTER 9 – IMPACT, CHALLENGES AND FUTURE ENHANCEMENTS....21
9.1 Impact Analysis....21
9.2 Challenges Faced....21
9.3 Future Enhancements....21
CHAPTER 10 – CONCLUSION....22
REFERENCES....23

LIST OF ABBREVIATIONS

Abbreviation	Full form
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
HTML	HyperText Markup Language
HTTP / HTTPS	HyperText Transfer Protocol / Secure
JS	JavaScript
OD	On Duty
QR	Quick Response (code)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
UI / UX	User Interface / User Experience

CHAPTER 1 – INTRODUCTION

1.1 Background

A college manages thousands of records every semester: student details, attendance, timetables, internal marks, fees, notices, events and requests. Traditionally these are handled through registers, files and separate spreadsheets. As students and staff increasingly use smartphones and web applications for everyday needs, a single digital platform for college management is a natural and practical improvement.

1.2 Overview of the Project

Campus Connect is a web-based system with three types of users: students, faculty and administrators. Students view their academic information and submit requests. Faculty record attendance and marks, post assignments and notices, and approve or reject leave and OD requests. Administrators manage users, departments, courses and timetables and monitor the whole campus from a dashboard. All users access the system through a browser on desktop or mobile.

1.3 Need for the Project

- Attendance and marks are recorded manually, which is slow and error-prone.
- Students do not get timely information on timetable changes, notices and results.
- Leave and OD requests travel on paper through several signatures.
- Data is scattered across files, so reports take a long time to prepare.
- There is no single place for students to raise grievances and track their status.

1.4 Scope of the Project

The scope covers a single college with three roles. The system handles registration and login, role-based dashboards, student and faculty profiles, department and course management, timetable, attendance, internal marks, assignments, notices, leave and OD requests, fee status display, grievances and reports. Online fee payment, multi-campus support and advanced analytics are kept as future work.

1.5 Organization of the Report

Chapter 2 states the problem and objectives. Chapter 3 compares the existing and proposed systems. Chapter 4 lists the requirements. Chapter 5 explains the system design. Chapter 6 describes the implementation and technology stack. Chapter 7 presents the results with screenshots. Chapter 8 covers testing. Chapter 9 discusses impact, challenges and future enhancements, followed by the conclusion and references.

CHAPTER 2 – PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Colleges rely on manual and disconnected processes for attendance, marks, timetables, notices and student requests. Students wait for information and approvals, faculty spend valuable teaching time on paperwork, and administrators struggle to collect accurate, up-to-date data. There is a need for a unified digital platform that connects students, faculty and administration, keeps information accurate and accessible, and speeds up everyday college processes.

2.2 Objectives

1. Provide a single, secure platform for students, faculty and administrators.
2. Digitise attendance marking and show students their attendance percentage.
3. Publish timetables, internal marks and assignments online.
4. Deliver notices and announcements instantly to the intended audience.
5. Simplify leave and OD requests with online approval and status tracking.
6. Provide a grievance module so students can raise and track issues.
7. Give administrators dashboards and reports for better decision making.
8. Ensure data security through hashed passwords and role-based access.

CHAPTER 3 – EXISTING SYSTEM AND PROPOSED SYSTEM

3.1 Existing System

In the existing system, attendance is taken in paper registers and totalled manually, marks are collected in files or spreadsheets, notices are pinned on boards or forwarded in messaging groups, and leave letters are signed physically. The limitations are:

- Time-consuming and error-prone manual record keeping.
- Delay in communicating notices, marks and timetable changes.
- No easy way for students to track their attendance percentage.
- Slow, paper-based approval of leave and OD.
- Difficult report generation and no central data store.

3.2 Proposed System

The proposed system replaces the manual flow with a digital workflow: Login, Role Dashboard, Operation (attendance, marks, notice, request), Approval, Notification and Report. It gives every user a personalised view of the information they need and gives administrators organised, searchable data.

3.3 Comparison

Aspect	Existing System	Proposed System
Attendance	Paper register, manual totals	Online marking, automatic percentage
Marks and results	Files and spreadsheets	Published online per student
Notices	Notice board / chat groups	Targeted online announcements
Leave / OD	Paper letter, multiple signatures	Online request with status tracking
Grievances	Verbal or written complaint	Online ticket with status
Reports	Prepared manually	Generated from the database
Security	No access control	Login and role-based access

Table 1: Comparison of existing and proposed systems

CHAPTER 4 – SYSTEM REQUIREMENTS

4.1 Functional Requirements

Student module

- Log in and log out securely; view and update profile.
- View timetable, attendance percentage and internal marks.
- View and submit assignments.
- Read notices and announcements.
- Apply for leave or OD and track approval status.
- Raise grievances and track their status; view fee status.

Faculty module

- Log in to the faculty dashboard; view assigned classes and timetable.
- Mark attendance and upload internal marks.
- Post assignments and notices for a class.
- Approve or reject leave and OD requests.
- Respond to grievances assigned to them.

Administrator module

- Add, edit, delete and search users (students and faculty).
- Manage departments, courses, classes and timetables.
- Publish college-wide announcements.
- View dashboard (total students, faculty, courses, requests) and reports.

4.2 Non-Functional Requirements

- **Usability:** simple, clean and responsive interface for desktop and mobile browsers.
- **Security:** session-based login, hashed passwords and role-based access.
- **Performance:** pages should load quickly even when many users log in together.
- **Reliability:** attendance, marks and requests must be stored correctly in the database.
- **Maintainability:** modular code hosted on GitHub for version control.

4.3 Hardware and Software Requirements

Item	Specification
Processor	Intel Core i3 or above
RAM	4 GB minimum (8 GB recommended)

Item	Specification
Operating system	Windows 10 / 11, Linux or macOS
Language	Python 3, HTML5, CSS3, JavaScript
Framework	Flask
Database	SQLite
Tools	Visual Studio Code, Git and GitHub, a web browser

Table 2: Hardware and software requirements

CHAPTER 5 – SYSTEM DESIGN

5.1 Solution Design

The system connects students, faculty and administrators through one digital platform. The student side covers viewing academic information and submitting requests. The faculty side covers attendance, marks, assignments, notices and approvals. The administrator side covers user, course and timetable management, announcements and reports. The key idea of the project is to bring all everyday college activities into a single, role-aware system.

5.2 System Architecture

The architecture has three layers. The front end (responsive web interface) is used by students, faculty and administrators. The back end application server contains modules for authentication and security, user management, department and course management, timetable, attendance, marks, assignments, notices, requests and grievances, and reporting, exposed through routes over HTTP/HTTPS. The database stores users, courses, timetable, attendance, marks, notices, requests and grievances.

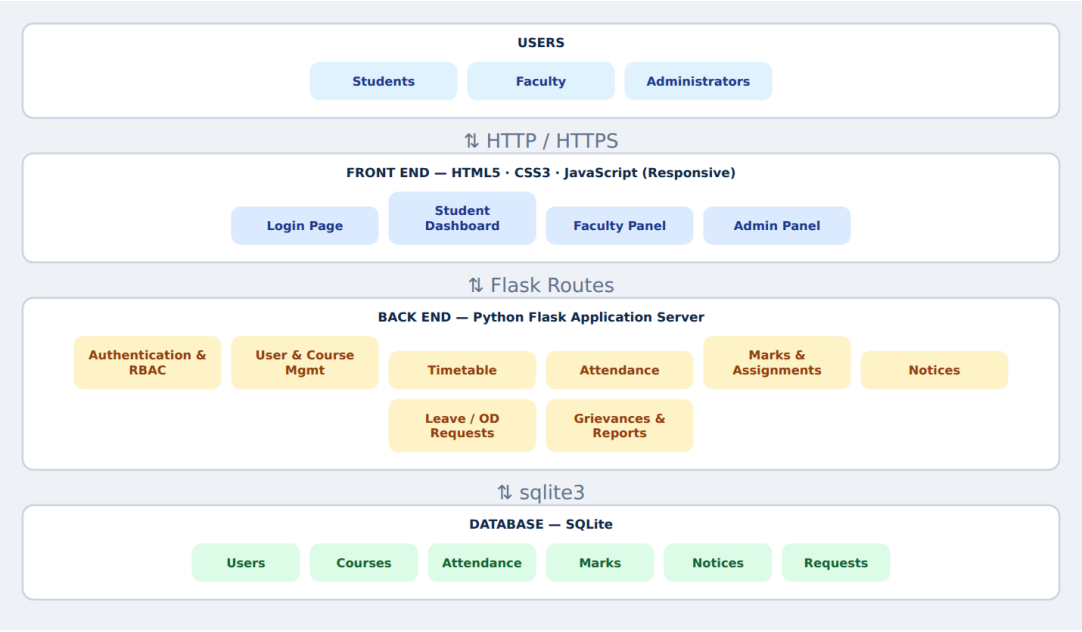


Figure 1: System architecture of the Campus Connect system

5.3 Workflow

Three flows are followed. In the **student workflow**, the student logs in, opens the dashboard, views timetable, attendance and marks, reads notices and submits a leave/OD request or grievance. In the **faculty workflow**, the faculty logs in, selects a class, marks attendance, uploads marks, posts assignments and notices, and approves or rejects requests.

In the **administrator workflow**, the admin logs in, manages users, departments, courses and timetables, publishes announcements and views reports.

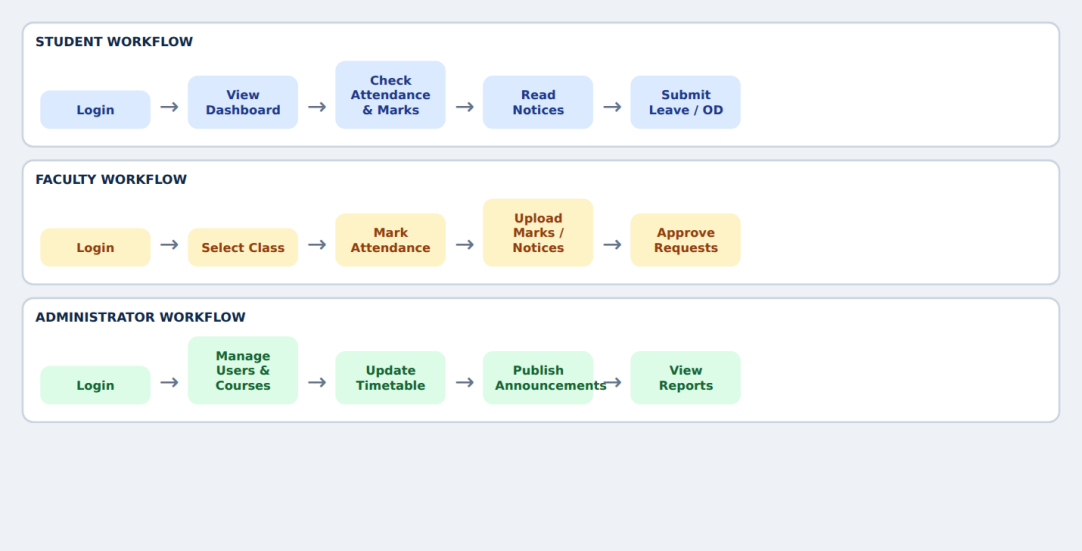


Figure 2: Student, faculty and administrator workflows

5.4 Module Description

- **User Management:** registration, login, logout, profile and role assignment.
- **Department and Course Management:** add, edit and delete departments, courses, subjects and classes.
- **Timetable Module:** create and display class-wise and faculty-wise timetables.
- **Attendance Module:** period-wise attendance marking and automatic percentage calculation.
- **Marks Module:** upload and display internal assessment marks.
- **Assignment Module:** post assignments with due dates and collect submissions.
- **Notice Module:** publish announcements to all users, a department or a class.
- **Request Module:** leave and OD requests with approval workflow and status.
- **Grievance Module:** raise, assign, respond to and close grievances.
- **Reports and Dashboard:** summary counts and attendance/marks reports for administrators.

5.5 Key Features

Secure role-based login, online attendance, marks and timetable, targeted notices, online leave/OD approval, grievance tracking, dashboards and reports.

CHAPTER 6 – IMPLEMENTATION

6.1 Technology Stack

Layer	Technology	Purpose
Frontend	HTML5	Structure of web pages
Styling	CSS3	UI design, layout and responsiveness
Client-side	JavaScript	Dynamic interactions and UI functionality
Backend	Python	Server-side application logic
Web framework	Flask	Routing, sessions, authentication and backend processing
Database	SQLite	Stores users, courses, attendance, marks, notices and requests
Database access	Python sqlite3	Connects the Flask application with SQLite
Authentication	Flask Session + Werkzeug	Login sessions and password hashing
Version control	Git and GitHub	Source-code management and hosting
Development tool	Visual Studio Code	Coding and project development

Table 3: Technology stack

6.2 Implementation Details

The application follows a client–server model. Flask routes handle requests for login, dashboards, attendance, marks, notices, requests and administration. Passwords are stored as hashes using Werkzeug and login state is maintained with Flask sessions. A decorator checks the user's role before each protected route so that students cannot open faculty or admin pages. SQLite stores data in tables for users, departments, courses, timetable, attendance, marks, assignments, notices, requests and grievances. The interface uses HTML5, CSS3 and JavaScript with a clean layout so that common tasks take only a few clicks.

The source code of the project is available on GitHub at [INSERT GITHUB LINK].

6.3 Student Side

After login, the student dashboard shows options such as My Profile, Timetable, Attendance, Marks, Assignments, Notices, Leave/OD and Grievances. Attendance is displayed subject-wise with the overall percentage, and a warning is shown when it falls below the required limit. Leave and OD requests show their current status (Pending, Approved or Rejected).

6.4 Faculty Side

The faculty panel lists the classes assigned to the faculty. Attendance is marked by selecting the class, subject and period and ticking present or absent for each student. Internal marks can be entered per student and are immediately visible to the student. Faculty can post assignments and notices and approve or reject requests with remarks.

6.5 Administrator Side

The admin panel has a sidebar with Dashboard, Users, Departments, Courses, Timetable, Announcements and Reports. The dashboard displays total students, faculty, courses and pending requests. Admins can search, add, edit and delete users and publish college-wide announcements.

6.6 AI and Smart Features (Proposed)

In the proposed design, simple analytics can identify students with low attendance or declining marks and alert the mentor early. Chatbot-based query support and smart notice summarisation are listed under future enhancements. The core management modules are implemented in the prototype.

CHAPTER 7 – RESULTS AND OUTPUT SCREENS

The following screenshots were taken from the working prototype.

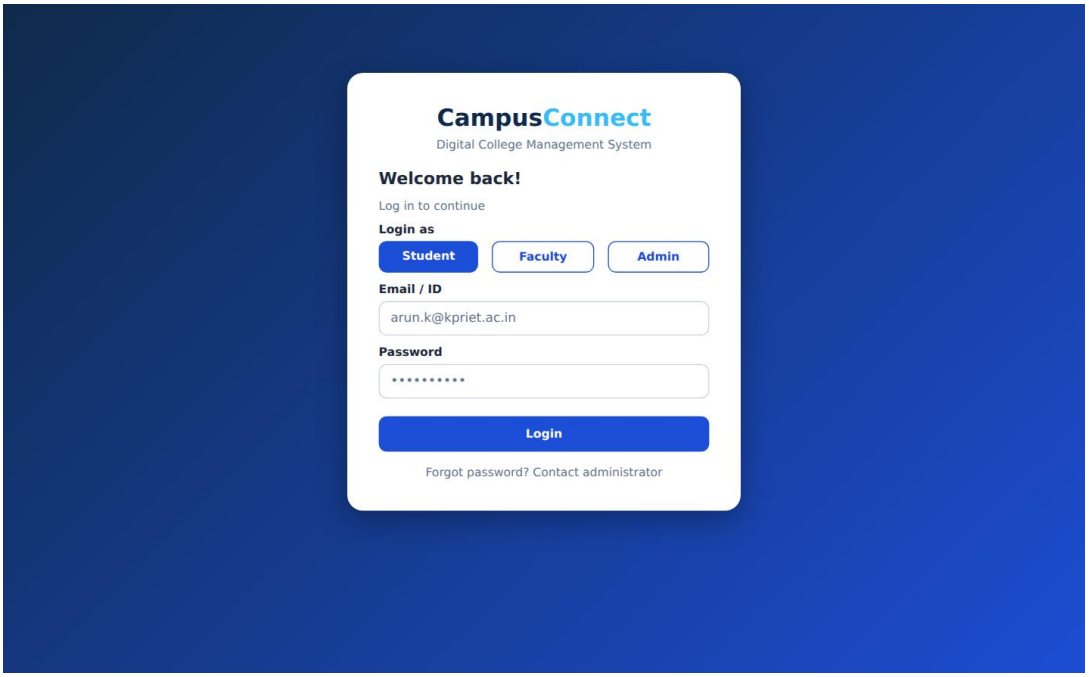


Figure 3: Login page

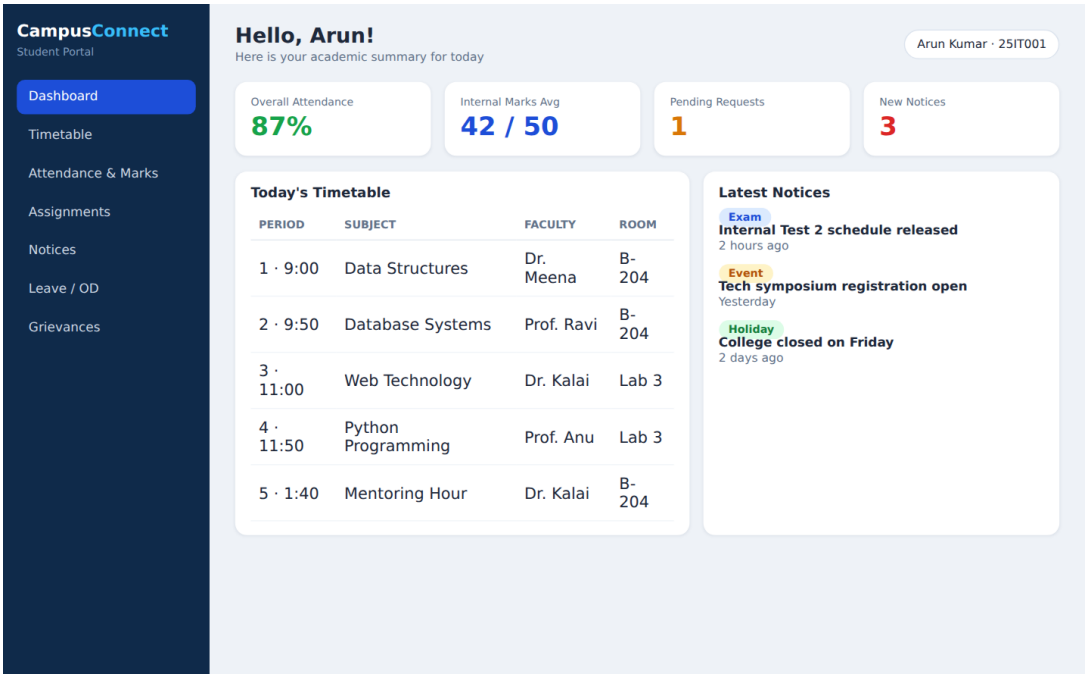


Figure 4: Student dashboard

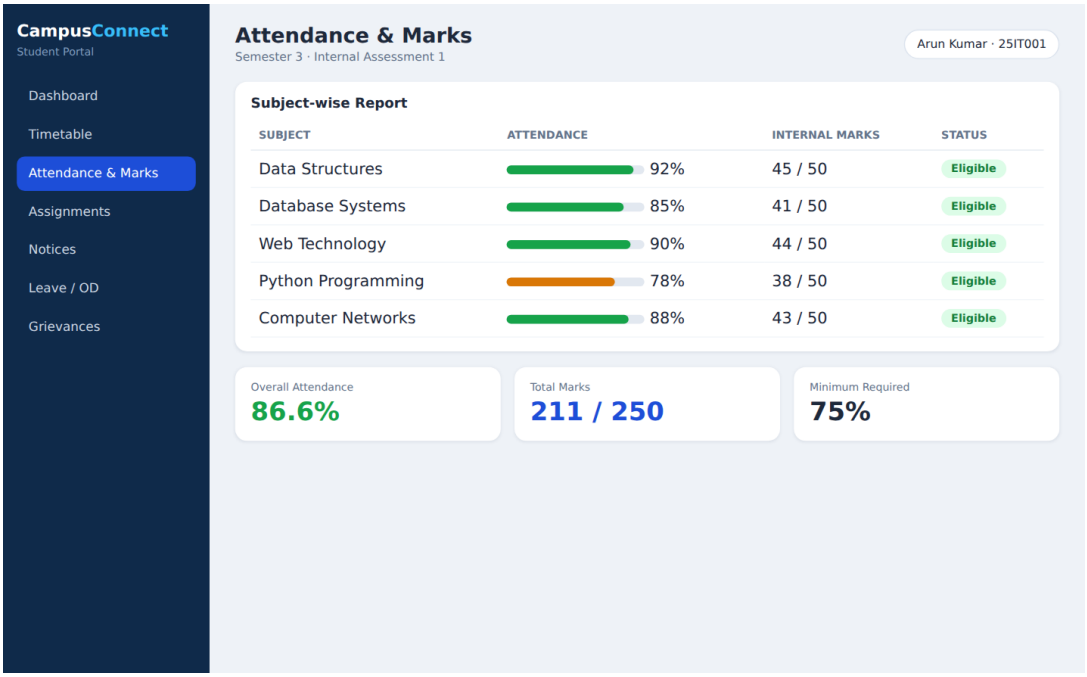


Figure 5: Attendance and marks view

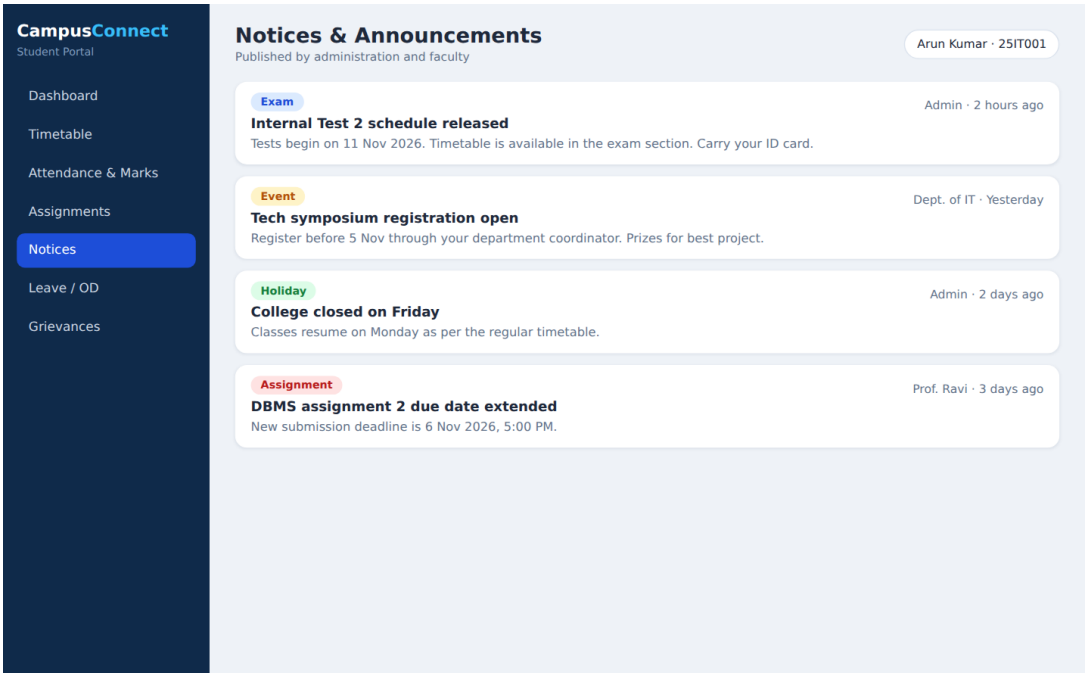


Figure 6: Notices page

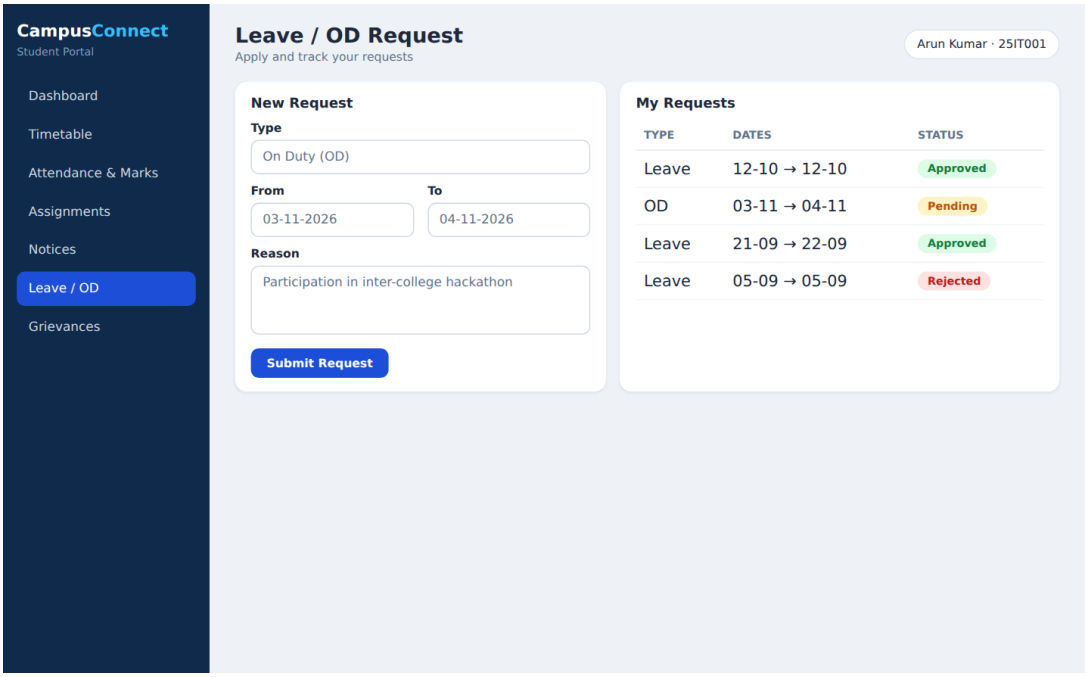


Figure 7: Leave / OD request page

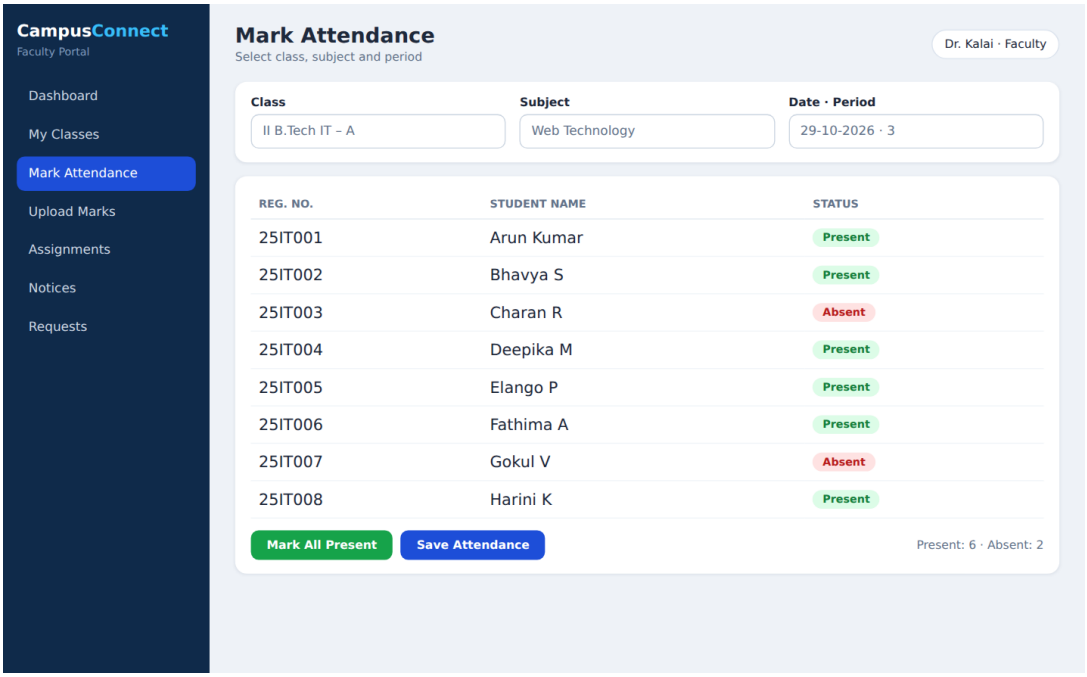


Figure 8: Faculty attendance marking page

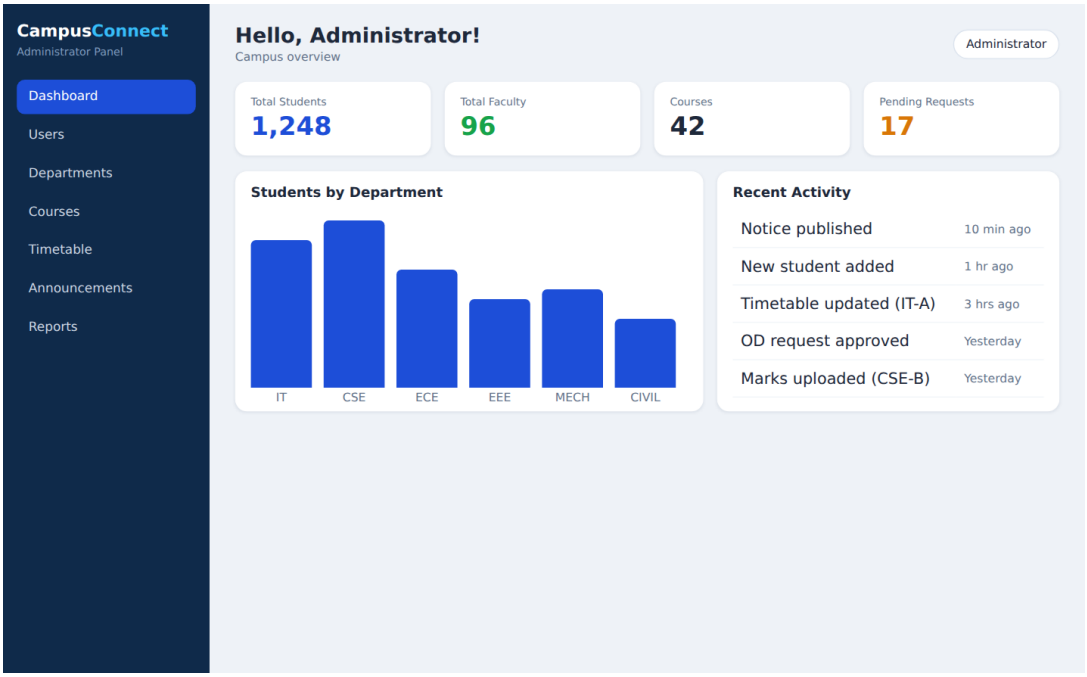


Figure 9: Admin dashboard

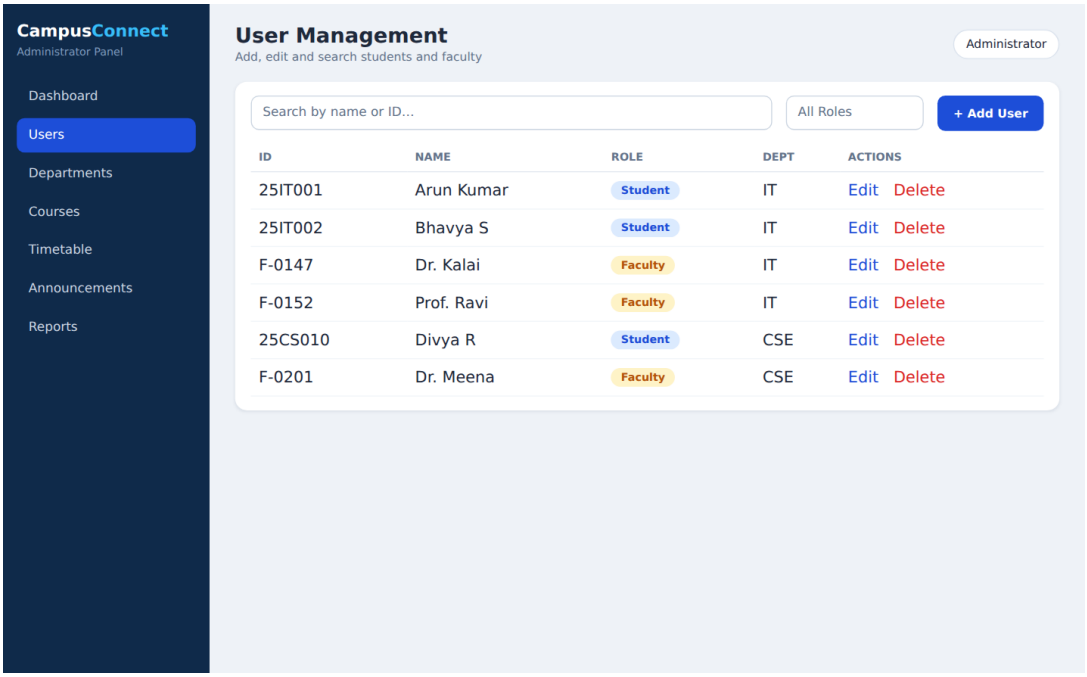


Figure 10: User management page of the admin panel

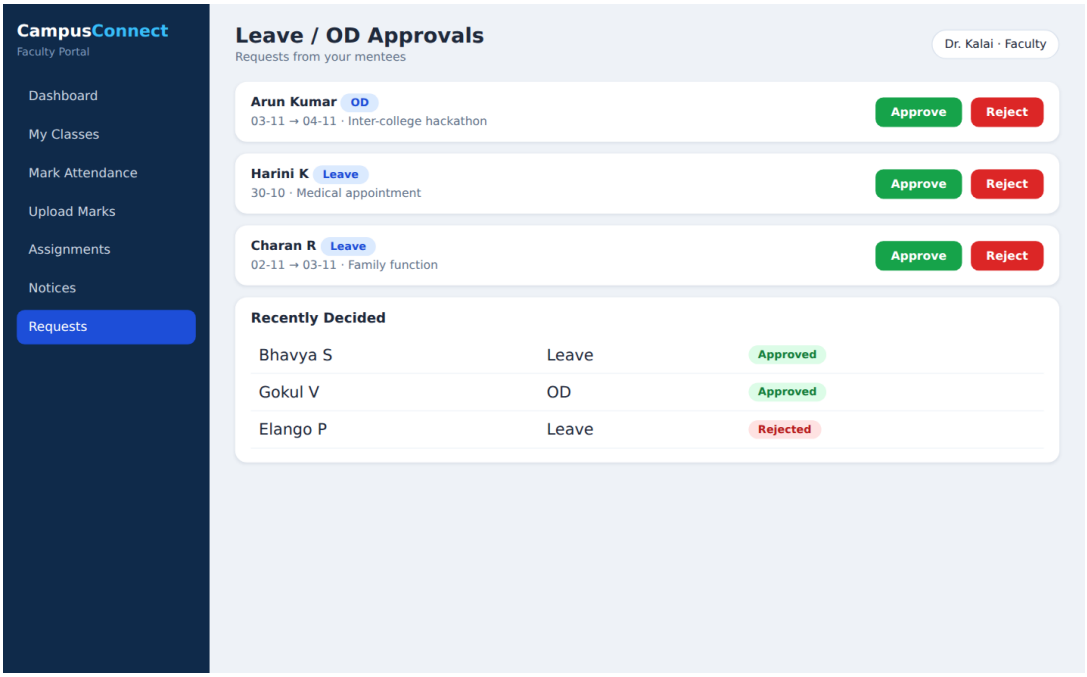


Figure 11: Request approval page

CHAPTER 8 – TESTING

8.1 Testing Approach

Functional (black-box) testing was carried out on each module. Every feature was checked with valid and invalid inputs and the results were compared with the expected output.

8.2 Test Cases

No.	Test case	Expected result	Result
1	Login with correct credentials	User reaches the role dashboard	Pass
2	Login with wrong password	Error message is shown	Pass
3	Student opens an admin URL	Access is denied	Pass
4	Faculty marks attendance	Attendance is saved and shown to student	Pass
5	Faculty uploads marks	Marks appear in the student's view	Pass
6	Admin publishes a notice	Notice is visible to the target users	Pass
7	Student submits leave request	Request appears as Pending	Pass
8	Faculty approves a request	Status changes to Approved	Pass
9	Admin adds or edits a user	User list is updated	Pass
10	Logout	Session ends and login page is shown	Pass

Table 4: Test cases

CHAPTER 9 – IMPACT, CHALLENGES AND FUTURE ENHANCEMENTS

9.1 Impact Analysis

- **Students:** instant access to attendance, marks, timetable and notices; easy online requests.
- **Faculty:** less paperwork, quick attendance and marks entry, faster approvals.
- **Administration:** organised data, quick reports and better monitoring.
- **College:** improved efficiency, transparency and digital transformation.

Before: registers, notice boards, paper letters, manual totals.

After: online attendance, targeted notices, online approvals, automatic reports.

9.2 Challenges Faced

- **Role-based access:** ensuring each user sees only what they are permitted to.
- **Data consistency:** keeping attendance, marks and timetable data accurate across modules.
- **Database design:** relating students, courses, faculty and classes correctly.
- **Usability:** designing one interface that suits three different kinds of users.
- **Approval workflow:** tracking request status from submission to decision.

9.3 Future Enhancements

- Online fee payment integration.
- Mobile application with push notifications.
- QR or face-recognition based attendance.
- AI-based early warning for low attendance and poor performance.
- Library, hostel and transport modules.
- Charts-based analytics and exportable reports.

CHAPTER 10 – CONCLUSION

Campus Connect provides a digital solution to reduce paperwork, delays and communication gaps in college administration. Students get timely access to attendance, marks, timetables and notices; faculty save time on routine tasks; and administrators gain organised data and reports. Role-based access and hashed passwords keep the data secure.

Working on this project helped us learn web development with Flask, database design with SQLite, UI design, teamwork and testing. The project can be extended with fee payment, a mobile app and AI-based analytics for wider use.

CONNECT | MANAGE | SIMPLIFY

REFERENCES

1. Flask Documentation, https://flask.palletsprojects.com
2. Python sqlite3 Documentation, https://docs.python.org/3/library/sqlite3.html
3. Werkzeug Security Utilities, https://werkzeug.palletsprojects.com
4. MDN Web Docs – HTML, CSS and JavaScript, https://developer.mozilla.org
5. SQLite Documentation, https://www.sqlite.org/docs.html
6. [Add your project presentation / flowchart reference here]